# Yet another causal model

Solve curse of Length

Fast inference performance

Low power Low cost

Beyond Transformer based GPT Model

# Yet another causal model

- **특징 ***

YAC Model 및 Hybrid Attention모델은 GPT Attention 모델과 비교하여 테스트 환경에서

GPU메모리 사용량, 학습 오차 수렴 속도, 수행 속도, 추론 정확도 모든면에서 성능 우위를 보이고

길이의 증가에 따라 이차적으로 비용이 증가하는 어텐션 모델과 달리 선형적으로 증가함으로서

대용량 LLM 구성시 월등한 성능우위를 나타낼 수 있다.

- **특징 ****

오토 리그레션 추론 과정에서 문장의 모든 토큰을 처음부터 매번 연산하는 것이 아닌

이전 문맥으로부터 다음 토큰을 바로 추론함으로써 최고 속도의 추론을 최저의 비용으로 수행한다.

- **특징 *****

길이 제약 없는 추론을 하드웨어 비용 증가 없이 저비용으로 수행할 수 있다.

# Not Transformer, Not Attention Model

YAC model는 어텐션(attention) 연산을 사용하지 않는 시계열 신경망 모델이다.

어텐션 연산은 시퀀스와 디멘젼의 증가에 따라 지수적으로 계산량이 증가하여 고비용 학습의 주 원인

현재까지 장기 의존성에 있어 자기주의(self-attention) 집중을 넘어서는 모델은 없었으나 본 모델은 어텐션 연산을 사용하지 않고 트랜스포머 GPT 대비 우수한 학습 수렴 속도 및 실행 속도, 메모리 저 사용량에서 대략 200% 정도의 우위를 보이고 이는 토큰량 및 시퀀스 길이의 증가에 따라 지수적인 차이를 발생시킨다.

시퀀스 길이와 차원 확대에 따른 계산비용이 선형적으로 증가하여 트랜스포머 대비 확장이 용이하고 비용이 대폭 감소되어 초 대용량 학습 및 추론에 제약이 없다.

비 언어 수치 시계열 데이터에서 어텐션 대비 우수한 추론 능력을 나타낸다.

# Yet another causal model

- **Solve curse of length**

문장 길이에 지수적으로 폭발하는 기존 트랜스포머 모델의 높은 메모리 및 연산 비용을 근본적이고 완벽하게 해결하여 완만한 선형적 하드웨어 증설로 제한 없는 토큰 컨택스트 윈도우 지원

길이의 저주를 해결함과 동시에 단위 환경에서 기존 트랜스포머 대비 학습 수렴 속도 및 추론 능력에서 200%의 우위 달성

- **Fast inference performance**

오토 리그레션 추론 과정에서 컨택스트의 모든 토큰을 처음부터 매번 연산하는 것이 아닌 이전 문맥으로부터 바로 추론함으로써 COT(Chain of Thought) 추론 과정에서 비 효율성을 제거하고 매우 적은 비용과 빠른 속도를 지원

## 비교

--ICLR 2025에 accept(spotlight)된 **"Hymba: A Hybrid-head Architecture for Small Language Models"** 논문--
Transformer와 Mamba block을 Parallel 하게 결합한 하이브리드 아키텍처로 Transformer의 높은 메모리 및 연산 비용과 Mamba의 성능 한계를 서로 보완하여, 효율성과 성능을 동시에 향상시킨 모델

Mamba의 성능 한계를 서로 보완 한다는 것은 지금까지 모든 트랜스포머의 고비용 문제를 개선한 모델은 학습 및 추론 성능 관점에서 트랜스포머를 능가하지 못했다는 것을 의미하고 하이브리드 모델의 성능 상한은 트랜스포머 architecture 이고 비용개선 반대 급부로서 필연적으로 트랜스포머 보다 성능 저하를 나타낼 수 밖에 없다는 사실을 보여준다.

본 Yac 모델은 기존 트랜스포머의 높은 계산 비용을 근원적으로 개선함과 동시에
모델 구조 자체로서 단위 환경에서 200%의 성능 우위를 나타내고 긴 컨택스트 윈도우 및 대량 코퍼스 학습에 비례하여 지수적 성능 향상을 제공한다.

# Pre-training 성능 비교

| | YAC Model | Hybrid Attention | GPT Attention |
|---|---|---|---|
| 100 에포크 학습 오차 | 0.674164 | 1.076424 | 1.118022 |
| 200 에포크 학습 오차 | 0.323494 | 0.403430 | 0.422084 |
| 100 에포크 학습 정확도 | 0.650000 | 0.141406 | 0.029687 |
| 200 에포크 학습 정확도 | 0.999219 | 0.807813 | 0.833594 |
| 메모리 사용률 | 64% | 67% | 90% |
| 1 에포크 학습 수행 시간 | 1분 14초 | 1분 20초 | 1분 42초 |
| 5 batch 추론 수행 시간 | 2초 | 3초 | 6초 |
| 문맥(context) 추론 | 지원 | 지원 | 미지원 |
| | | | |

GPU: RTX3090 Memory: 24G

Pre-trainning 학습 구성: 512 길이단위 chunk, 32 batch

추론 정확도 측정: 512 길이의 전반부 입력으로주고 후반부 일치 스코어 계산

# 증류 데이터 셋 학습 결과

| | YAC Model | GPT Attention |
|---|---|---|
| Alpaca 학습 오차(100 epoch) | Train Loss: 1.7548, Val Loss: 1.8900, Val Acc: 0.3931 | Train Loss: 1.2440, Val Loss: 1.8043, Val Acc: 0.4401 |
| Alpaca 추론 정확도(100 epoch) | train(A): 0.144522, test(B): 0.088328[max] | train(A): 0.076739, test(B): 0.044118[max] |
| Databricks dolly 학습 오차(30 epoch) | Train Loss: 0.7084, Val Loss: 6.6170, Val Acc: 0.2682 | Train Loss: 1.1986, Val Loss: 5.6136, Val Acc: 0.3005 |
| Databricks dolly 추론 정확도(30)<br>Databricks dolly 추론 정확도(100) | train(A): 0.727273, test(B): 0.060201 | train(A): 0.195556, test(B): 0.037209<br>train(A): 0.680000, test(B): 0.041860[arbitrary max] |
| 학습 오차 해석 | 전체 맥락 학습 인해 point wise 학습 오차 수렴이 GPT 보다 느림 | point wise 학습으로 인해 학습 오차 수렴이 빠름 |
| 추론 정확도 해석 | 전체 맥락 학습 인해 문장 단위 추론 정확도 향상 | point wise 학습으로 인해 문장 단위 추론 정확도가 Yac 대비 낮음 |
| Alpaca 와 dolly 학습 데이터 해석 | Dolly는 **사람이 쓴 이질적·비균질 데이터**라서 **토큰 단위 일반화(PPL)**의 난도가 높고, 과적합/분포불일치에 취약, 데이터 다양성·전역 일관성 요구 | OpenAI에 의한 합성-교사 데이터(Alpaca)는 **형식·스타일이 균질**하여 **토큰 단위 교차엔트로피 최적화(GPT)**와 잘 맞음 |
| 과적합 | YAC은 문맥 전체를 대상으로 하는 확률 분포를 사용하기 때문에, 개별 토큰의 빈도나 지역적 패턴에 과적합되기 어렵습니다, loss가 안정적으로 수렴하거나 거의 정체됩니다, loss가 더 줄지 않는다"는 것은 오히려 **일반화(generalization)가 잘 유지되고 있다**는 신호일 수 있습니다 | point wise 학습으로 인해 학습 오차가 Yac대비 좀더 수렴하나 문장 단위 추론 정확도, validation accuracy가 낮음(즉, 과적합 신호)이 나타남 |
| 할루시네이션 | 전체 맥락 학습 인해 모델 자체 특성으로 인한 환각 완화 가능성 | point wise 학습으로 인해 환각에 취약 |

# Sign Curve ( 비 언어 수치 데이터 예측 정확도 비교 )

- 테스트 데이터의 전체 길이의 반을 입력으로 주고 나머지 반을 auto regression으로 예측

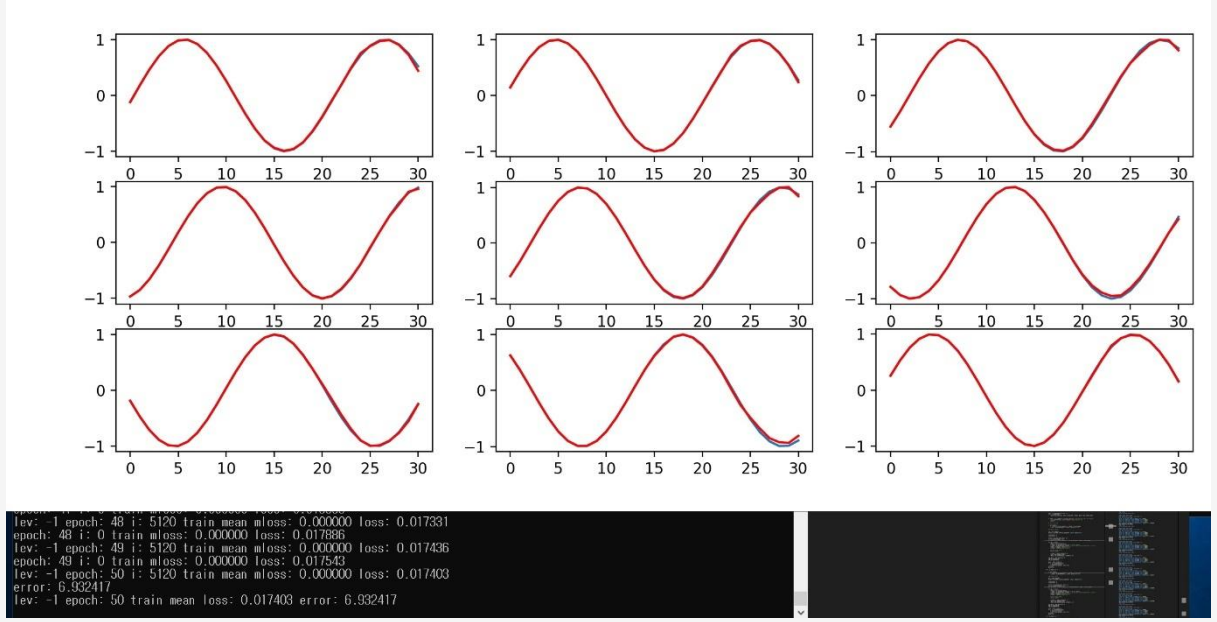
- GPT 와 비교하여 테스트 데이터 셋 추론에서 그래프 육안 식별 과 평균제곱 오차 모두 YAC model 이 정확

- 이와 같이 규칙성이 명확한 케이스는 정확한 학습 및 추론 이 되야 하나 학습 오차를 보 면 트랜스포머 계열 모델은 오차가 계속 수렴하지 않고 진동하며 정확도가 떨어지고 테스트 셋 실험 결과 그래프 역시 육안으로도 미세한 오차가 확인된다. 이에 반해 YAC model 모델은 오차가 계속 수렴하며 정밀한 추론 결과를 보여준다.

- Red는 예측값, blue는 정답, GPT의 경우 blue가 약간 나타나나 YAC model 의 경우 완전히 오버랩되어 red만 보여짐.

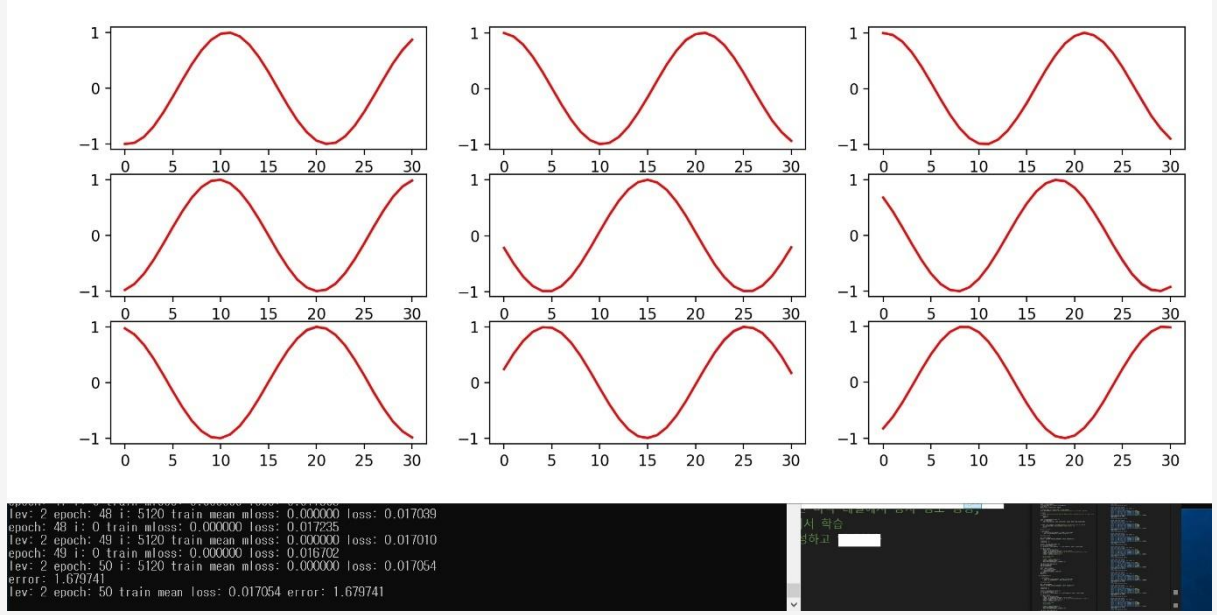
# Sign Curve ( 비 언어 수치 데이터 예측 정확도 비교 그래프 )

GPT




Error: 6.94471

YAC




Error: 1.679741

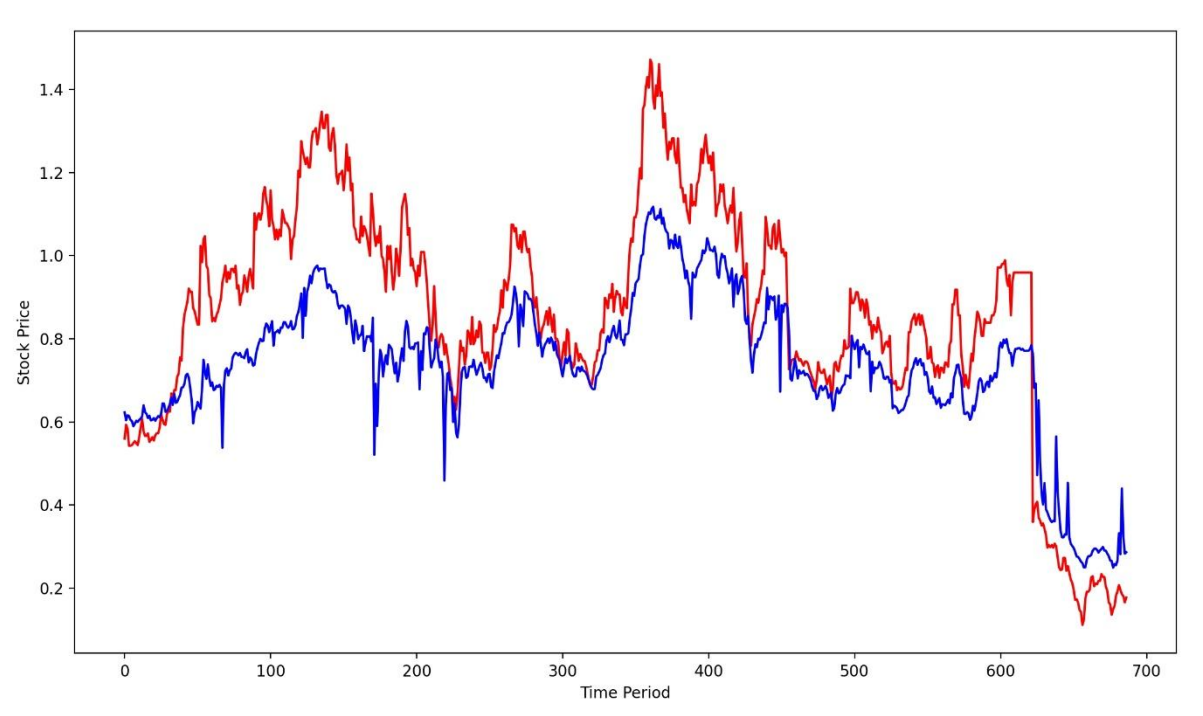
# 주가 예측 ( 비 언어 수치 데이터 추론 능력 비교 )

- 과거 추세 데이터로 학습 및 예측하는 케이스에 있어 일반적인 경우와 달리 입력 데이터에 목표 주가에 관한 정보와 미래 값을 포함하지 않아 추종할(lagging) 입력값이 없어 예측이 어려운 케이스로 학습하여 다음 날 종가를 예측한다.

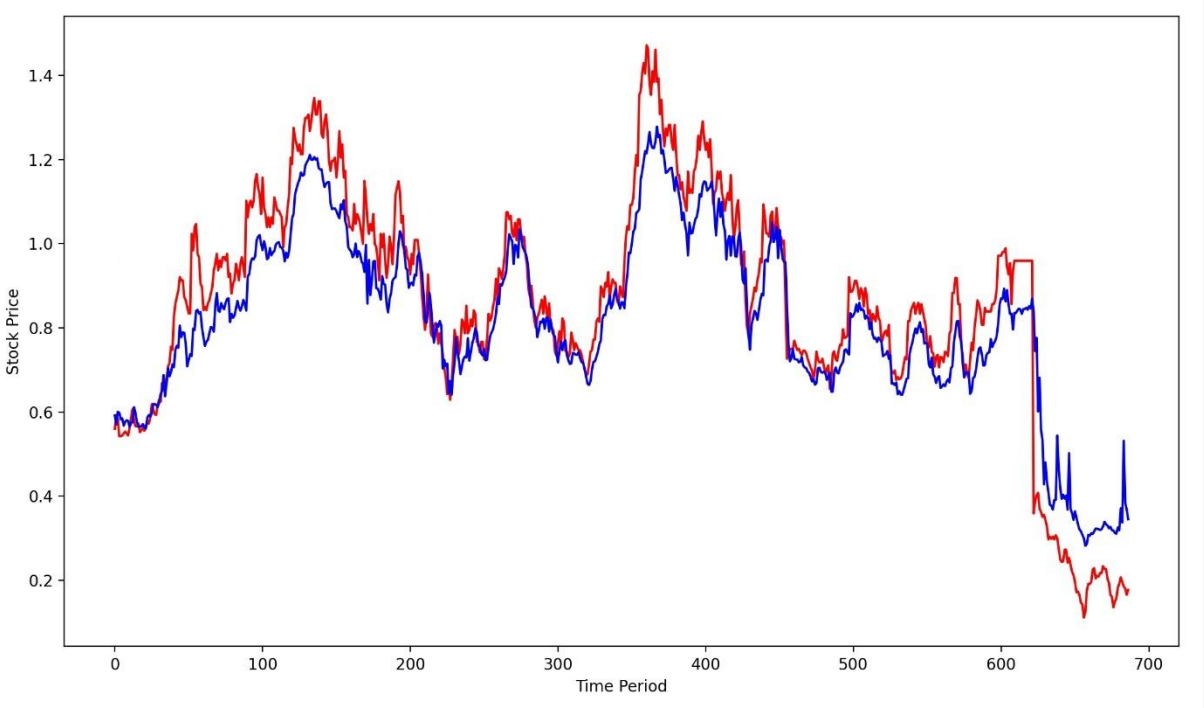
- GPT 의 경우보다 목표치에 더 근접한 추론 결과를 나타낸다.

# 주가 예측 ( 비 언어 수치 데이터 추론 능력 비교 그래프 )

GPT

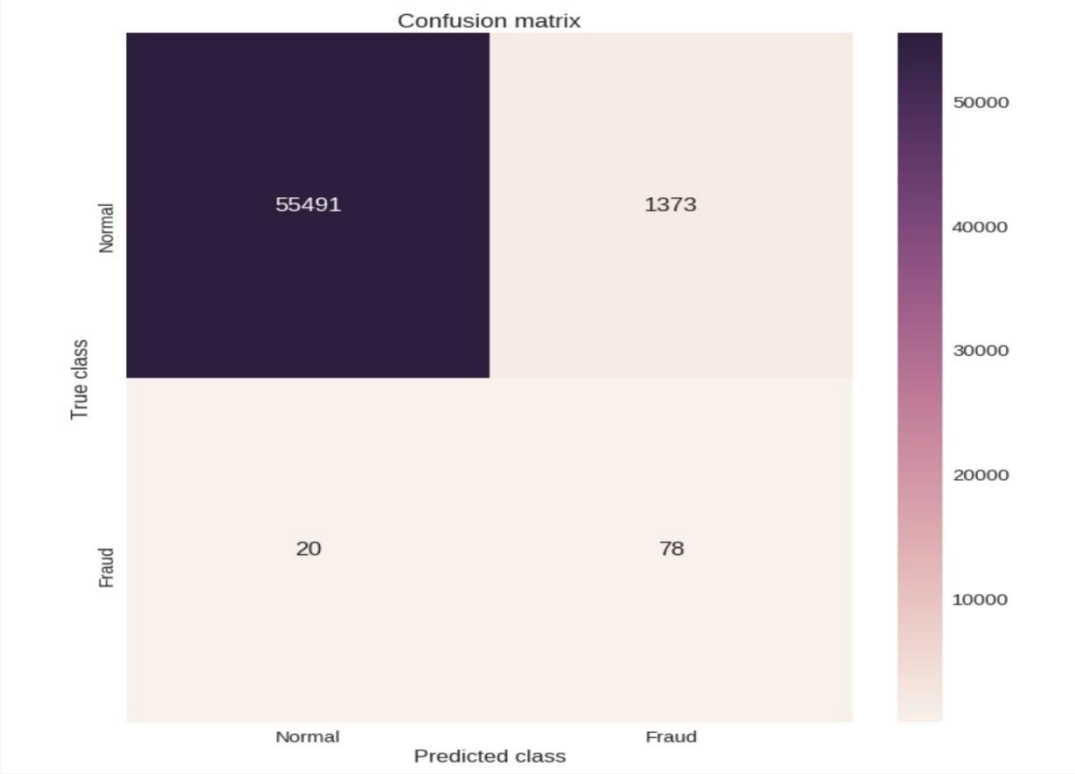



YAC

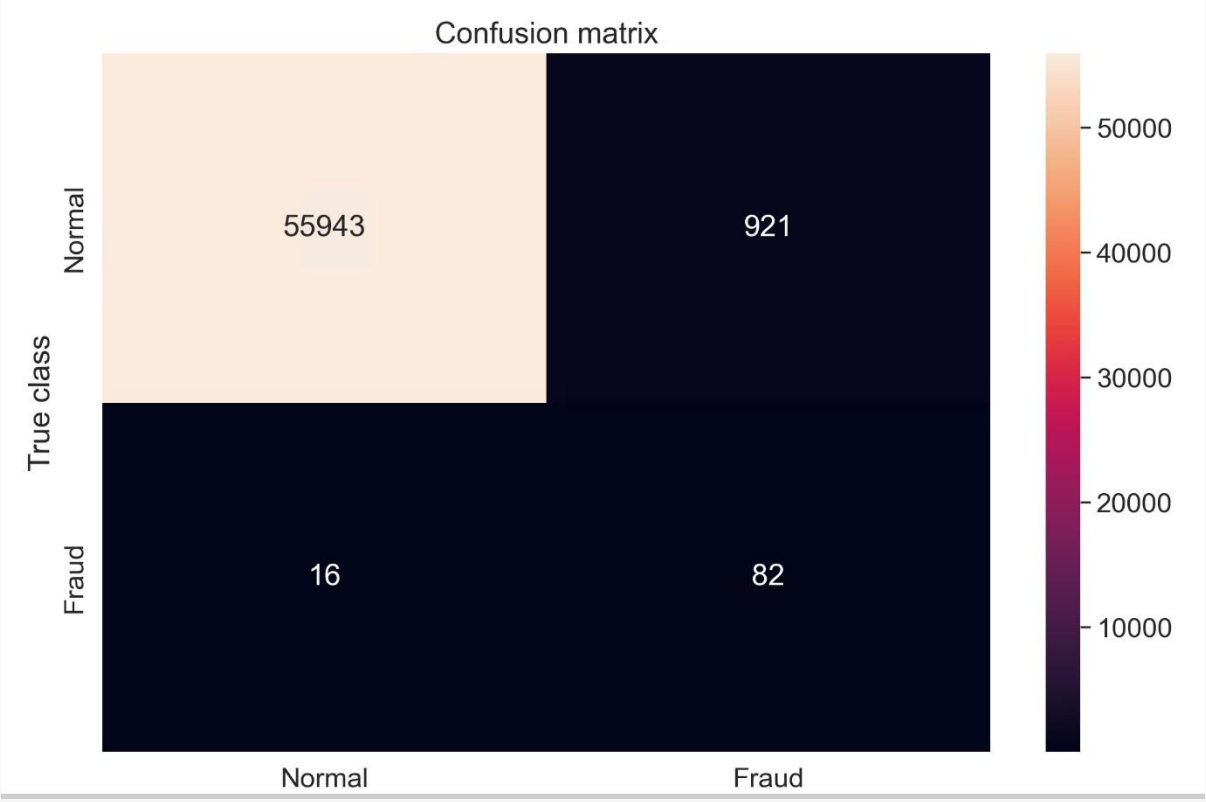


# Anormal Detection ( 이상탐지, 희소 데이터 학습 추론 능력 )

## Auto Encoder




## YAC




좌상단에서 우하단 대각선 방향의 값들은 크고

우상단에서 좌하단 대각선 방향의 값들은 작을 수록 탐지 능력이 좋음

세로방향: 정답 class

가로방향: 예측 class

# 감사합니다.

모든 예제는 깃허브: mempute (github.com)

Ai framework: mempute/flux: AI framework, c++ & python 1: 1 matching, API supporting host and gpu(cuda) libraries, powerfull time series neural net (github.com)

LLM: mempute/v4: Time series models built with PyTorch C++ and supporting Python interfaces, including: Spiking model using som(self-organizing map), quantum attention model, Bayesian inference network using rdbms (github.com)

**LLM 사전 학습 결과:** mempute/learn_results (github.com)